COMPILER-STEPS-without-critics

Compiler pipeline, step by step — with no critics

Both critics disabled. This is the fallback the experiment used whenever a task exhausted its three attempts with the critics enabled. Extracted verbatim from `logs/run-*.log` for the three runs behind `docs/artifacts/arm-fm-vs-compiler/` (provider `opencode`, model `mimo-v2.6-flash`).

Steps 1, 3, 4 and 5 are unchanged: the critics only gate the proposal and the finished machine, they never touch LTLf, the DFA, or the Reward Machine. Steps 2 and 6 are skipped.

Tasks 1, 3 and 4 are the fallback runs from the experiment itself (they exhausted three attempts with the critics enabled). Task 2 passed on its first attempt with the critics, so the experiment never disabled them for it; a supplementary critics-off run of task 2 was added to complete the file.

Task 1 — `Use the key to open the door, then reach the goal.`

Identical in all three runs (run 1 shown). Outcome: **accepted** on attempt 1. Source: `logs/run-20260922T184341.891732Z.log`.

1. Validated proposal (clauses)


```
{
  "clauses": [
    {
      "normalized_clause": "Precedence(has_key, door_open)",
      "pattern": "Precedence",
      "priority": "hard",
      "propositions": [
        "has_key",
        "door_open"
      ]
    },
    {
      "normalized_clause": "Precedence(door_open, at_goal)",
      "pattern": "Precedence",
      "priority": "hard",
      "propositions": [
        "door_open",
        "at_goal"
      ]
    }
  ],
  "task": "Use the key to open the door, then reach the goal."
}
```


2. Task critic

```
skipped (disabled for this run)
```

3. LTLf formulas

```
[{'formula': '((~door_open)U(has_key&((~door_open)&(X(Fdoor_open)))))',
  'normalized_clause': 'Precedence(has_key, door_open)'},
 {'formula': '((~at_goal)U(door_open&((~at_goal)&(X(Fat_goal)))))',
  'normalized_clause': 'Precedence(door_open, at_goal)'}]
```

4. MONA DFA

```
({'accepting_states': {'S4'},
  'alphabet': {'door_open', 'has_key'},
  'initial_state': 'S1',
  'states': {'S3', 'S1', 'S4', 'S2'},
  'transitions': {('S1', '!door_open&!has_key'): 'S1',
                  ('S1', '!door_open&has_key'): 'S2',
                  ('S1', 'door_open'): 'S3',
                  ('S2', '!door_open'): 'S2',
                  ('S2', 'door_open'): 'S4',
                  ('S3', ''): 'S3',
                  ('S4', ''): 'S4'}},
 {'accepting_states': {'S4'},
  'alphabet': {'door_open', 'at_goal'},
  'initial_state': 'S1',
  'states': {'S3', 'S1', 'S4', 'S2'},
  'transitions': {('S1', '!at_goal&!door_open'): 'S1',
                  ('S1', '!at_goal&door_open'): 'S2',
                  ('S1', 'at_goal'): 'S3',
                  ('S2', '!at_goal'): 'S2',
                  ('S2', 'at_goal'): 'S4',
                  ('S3', ''): 'S3',
                  ('S4', ''): 'S4'}})
```

5. Reward Machine (as compiled)

```
s: 0, 1, 2, 3
i: 0
f: 4
r: 0
0; 1; !has_key,!door_open,at_goal; 0
0; 1; !has_key,door_open,!at_goal; 0
0; 1; !has_key,door_open,at_goal; 0
0; 2; has_key,!door_open,!at_goal; 0
0; 1; has_key,!door_open,at_goal; 0
0; 1; has_key,door_open,!at_goal; 0
0; 1; has_key,door_open,at_goal; 0
2; 1; !has_key,!door_open,at_goal; 0
2; 3; !has_key,door_open,!at_goal; 0.10
2; 1; !has_key,door_open,at_goal; 0
2; 1; has_key,!door_open,at_goal; 0
2; 3; has_key,door_open,!at_goal; 0.10
2; 1; has_key,door_open,at_goal; 0
3; 4; !has_key,!door_open,at_goal; 1.10
3; 4; !has_key,door_open,at_goal; 1.10
3; 4; has_key,!door_open,at_goal; 1.10
3; 4; has_key,door_open,at_goal; 1.10
```

The same machine in the ARM-FM paper syntax:

```
REWARD_MACHINE:
STATES: u0, u1, u2, u3, u4
INITIAL_STATE: u0
FINAL_STATES: u4
DEFAULT_REWARD: 0
TRANSITION_FUNCTION:
(u0, !has_key & !door_open & at_goal) -> u1
(u0, !has_key & door_open & !at_goal) -> u1
(u0, !has_key & door_open & at_goal) -> u1
(u0, has_key & !door_open & !at_goal) -> u2
(u0, has_key & !door_open & at_goal) -> u1
(u0, has_key & door_open & !at_goal) -> u1
(u0, has_key & door_open & at_goal) -> u1
(u2, !has_key & !door_open & at_goal) -> u1
(u2, !has_key & door_open & !at_goal) -> u3
(u2, !has_key & door_open & at_goal) -> u1
(u2, has_key & !door_open & at_goal) -> u1
(u2, has_key & door_open & !at_goal) -> u3
(u2, has_key & door_open & at_goal) -> u1
(u3, !has_key & !door_open & at_goal) -> u4
(u3, !has_key & door_open & at_goal) -> u4
(u3, has_key & !door_open & at_goal) -> u4
(u3, has_key & door_open & at_goal) -> u4
REWARD_FUNCTION:
(u0, !has_key & !door_open & at_goal, u1) -> 0
(u0, !has_key & door_open & !at_goal, u1) -> 0
(u0, !has_key & door_open & at_goal, u1) -> 0
(u0, has_key & !door_open & !at_goal, u2) -> 0
(u0, has_key & !door_open & at_goal, u1) -> 0
(u0, has_key & door_open & !at_goal, u1) -> 0
(u0, has_key & door_open & at_goal, u1) -> 0
(u2, !has_key & !door_open & at_goal, u1) -> 0
(u2, !has_key & door_open & !at_goal, u3) -> 0.1
(u2, !has_key & door_open & at_goal, u1) -> 0
(u2, has_key & !door_open & at_goal, u1) -> 0
(u2, has_key & door_open & !at_goal, u3) -> 0.1
(u2, has_key & door_open & at_goal, u1) -> 0
(u3, !has_key & !door_open & at_goal, u4) -> 1.1
(u3, !has_key & door_open & at_goal, u4) -> 1.1
(u3, has_key & !door_open & at_goal, u4) -> 1.1
(u3, has_key & door_open & at_goal, u4) -> 1.1
```

6. RM critic

```
skipped (disabled for this run)
```

Task 2 — `Eventually acquire the key.`

Supplementary run. The experiment's three recorded runs never disabled the critics for this task, because it was accepted on its first attempt with them enabled. This extra run was made to complete the file. Outcome: **accepted** on attempt 1. Source: `logs/run-20260928T163805.885472Z.log` .

The clauses, formula, and machine below are **identical** to the critics-on run of the same task — for a single-clause `Existence` task the critics change nothing.

1. Validated proposal (clauses)


```
{
  "clauses": [
    {
      "normalized_clause": "Existence(has_key)",
      "pattern": "Existence",
      "priority": "none",
      "propositions": [
        "has_key"
      ]
    }
  ],
  "task": "Eventually acquire the key."
}
```


2. Task critic

```
skipped (disabled for this run)
```

3. LTLf formulas


```
[{'formula': '(Fhas_key)', 'normalized_clause': 'Existence(has_key)'}]
```


4. MONA DFA


```
({'accepting_states': {'S2'},
  'alphabet': {'has_key'},
  'initial_state': 'S1',
  'states': {'S2', 'S1'},
  'transitions': {('S1', '!has_key'): 'S1', ('S1', 'has_key'): 'S2', ('S2', ''): 'S2'}},)
```


5. Reward Machine (as compiled)

```
s: 0
i: 0
f: 1
r: 0
0; 1; has_key; 1.10
```

The same machine in the ARM-FM paper syntax:

```
REWARD_MACHINE:
STATES: u0, u1
INITIAL_STATE: u0
FINAL_STATES: u1
DEFAULT_REWARD: 0
TRANSITION_FUNCTION:
(u0, has_key) -> u1
REWARD_FUNCTION:
(u0, has_key, u1) -> 1.1
```

6. RM critic

```
skipped (disabled for this run)
```

Task 3 — `Open the door only after acquiring the key.`

Identical in all three runs (run 1 shown). Outcome: **accepted** on attempt 1. Source: `logs/run-20260922T184440.974752Z.log`.

1. Validated proposal (clauses)


```
{
  "clauses": [
    {
      "normalized_clause": "Precedence(has_key, door_open)",
      "pattern": "Precedence",
      "priority": "hard",
      "propositions": [
        "has_key",
        "door_open"
      ]
    }
  ],
  "task": "Open the door only after acquiring the key."
}
```


2. Task critic

```
skipped (disabled for this run)
```

3. LTLf formulas

```
[{'formula': '((~door_open)U(has_key&((~door_open)&(X(Fdoor_open)))))',
  'normalized_clause': 'Precedence(has_key, door_open)'}]
```

4. MONA DFA

```
({'accepting_states': {'S4'},
  'alphabet': {'door_open', 'has_key'},
  'initial_state': 'S1',
  'states': {'S3', 'S1', 'S4', 'S2'},
  'transitions': {('S1', '!door_open&!has_key'): 'S1',
                  ('S1', '!door_open&has_key'): 'S2',
                  ('S1', 'door_open'): 'S3',
                  ('S2', '!door_open'): 'S2',
                  ('S2', 'door_open'): 'S4',
                  ('S3', ''): 'S3',
                  ('S4', ''): 'S4'}},)
```

5. Reward Machine (as compiled)

```
s: 0, 1, 2
i: 0
f: 3
r: 0
0; 1; !has_key,door_open; 0
0; 2; has_key,!door_open; 0
0; 1; has_key,door_open; 0
2; 3; !has_key,door_open; 1.10
2; 3; has_key,door_open; 1.10
```

The same machine in the ARM-FM paper syntax:

```
REWARD_MACHINE:
STATES: u0, u1, u2, u3
INITIAL_STATE: u0
FINAL_STATES: u3
DEFAULT_REWARD: 0
TRANSITION_FUNCTION:
(u0, !has_key & door_open) -> u1
(u0, has_key & !door_open) -> u2
(u0, has_key & door_open) -> u1
(u2, !has_key & door_open) -> u3
(u2, has_key & door_open) -> u3
REWARD_FUNCTION:
(u0, !has_key & door_open, u1) -> 0
(u0, has_key & !door_open, u2) -> 0
(u0, has_key & door_open, u1) -> 0
(u2, !has_key & door_open, u3) -> 1.1
(u2, has_key & door_open, u3) -> 1.1
```

6. RM critic

```
skipped (disabled for this run)
```

Task 4 — `Reach the goal only after opening the door.`

Identical in all three runs (run 1 shown). Outcome: **accepted** on attempt 1. Source: `logs/run-20260922T184518.102392Z.log`.

1. Validated proposal (clauses)


```json
{
  "clauses": [
    {
      "normalized_clause": "Precedence(door_open, at_goal)",
      "pattern": "Precedence",
      "priority": "hard",
      "propositions": [
        "door_open",
        "at_goal"
      ]
    }
  ],
```

```
  "task": "Reach the goal only after opening the door."
}
```

2. Task critic

```
skipped (disabled for this run)
```

3. LTLf formulas

```
[{'formula': '((~at_goal)U(door_open&((~at_goal)&(X(Fat_goal))))))',
  'normalized_clause': 'Precedence(door_open, at_goal)'}]
```

4. MONA DFA

```
({'accepting_states': {'S4'},
  'alphabet': {'door_open', 'at_goal'},
  'initial_state': 'S1',
  'states': {'S3', 'S1', 'S4', 'S2'},
  'transitions': {('S1', '!at_goal&!door_open'): 'S1',
                  ('S1', '!at_goal&door_open'): 'S2',
                  ('S1', 'at_goal'): 'S3',
                  ('S2', '!at_goal'): 'S2',
                  ('S2', 'at_goal'): 'S4',
                  ('S3', ''): 'S3',
                  ('S4', ''): 'S4'}},)
```

5. Reward Machine (as compiled)

```
s: 0, 1, 2
i: 0
f: 3
r: 0
0; 1; !door_open,at_goal; 0
0; 2; door_open,!at_goal; 0
0; 1; door_open,at_goal; 0
2; 3; !door_open,at_goal; 1.10
2; 3; door_open,at_goal; 1.10
```

The same machine in the ARM-FM paper syntax:

```
REWARD_MACHINE:
STATES: u0, u1, u2, u3
INITIAL_STATE: u0
FINAL_STATES: u3
DEFAULT_REWARD: 0
TRANSITION_FUNCTION:
(u0, !door_open & at_goal) -> u1
(u0, door_open & !at_goal) -> u2
(u0, door_open & at_goal) -> u1
(u2, !door_open & at_goal) -> u3
(u2, door_open & at_goal) -> u3
REWARD_FUNCTION:
(u0, !door_open & at_goal, u1) -> 0
(u0, door_open & !at_goal, u2) -> 0
```

```
(u0, door_open & at_goal, u1) -> 0
(u2, !door_open & at_goal, u3) -> 1.1
(u2, door_open & at_goal, u3) -> 1.1
```

6. RM critic

```
skipped (disabled for this run)
```